



FILIA UNIwersYTETU W BIAŁYMSTOKU W WILNIE
KIERUNEK: INFORMATYKA

Daniel Subatovič

Nr albumu: 91629

Kalendarz wydarzeń kulturalnych

Wilno 2025

Pusta strona w przypadku obustronnego drukowania

SPIS TREŚCI

WSTĘP.....	4
ROZDZIAŁ 1. Teoria Algorytmów.....	5
1.1 Opis działania Tabeli	5
1.2 Algorytmy zachowania danych do Csv	6
1.3 Algorytmy sortowania.....	7
1.4 Algorytmy filtrowania i wyszukiwania.....	9
1.5 Inne algorytmy	11
1.6 Obsługa danych.....	13
ROZDZIAŁ 2. Projekt systemu	15
2.1 Początek działania programu	15
2.2 Działanie programu.....	16
ROZDZIAŁ 3. Implementacja	21
3.1 Opis kroków implementacji	21
ROZDZIAŁ 4. Testowanie i analiza wyników	22
4.1 Testy i ich wyniki.....	22
ROZDZIAŁ 5. Wnioski	24
5.1 Ocena implementacji.....	24
ZAKOŃCZENIE	25
Spis rysunków	26
Załącznik: Kod Programu	28

WSTĘP

W ramach projektu otrzymałem do zaimplementowania program, który reprezentuje działanie kalendarza zdarzeń kulturalnych.

Celem pracy jest stworzenie prostej aplikacji do ewidencji lokalnych wydarzeń kulturalnych z możliwością planowania dnia/tygodnia. Filtrowania/wyszukiwania, a także wykrywania konfliktów czasowych.

Praca składa się z pięciu rozdziałów. **W pierwszym rozdziale** będzie przedstawiony szczegółowy opis działania algorytmu, omówienie jego zastosowań i opis struktury danych.

Rozdział drugi zostanie poświęcony architekturze projektu.

W trzecim rozdziale będzie omówiona jego implementacja.

Rozdział czwarty przedstawi testy i wyniki tego programu.

W ostatnim rozdziale zostanie przekazana ocena implementacji algorytmu i jego wydajności.

ROZDZIAŁ 1. TEORIA ALGORYTMÓW

Kalendarz zdarzeń kulturalnych zawiera wiele różnych algorytmów. W tym rozdziale będzie przedstawione ich działanie.

1.1 Opis działania Tabeli

Po włączeniu programu od razu zostają odpalone wszystkie funkcje wpisane dane do tabeli za pomocą `LoadDataOnStartup()`; , gdzie o sprawdzeniu zostają załadowane dane do oryginalnej klasy `events`. Inne klasy są używane w filtracji lub sortowaniu.

Rysunek 1. Load data on start

```
private void LoadDataOnStartup()
{
    string fileName = "data.csv";

    if (!File.Exists(fileName))
        return;

    try
    {
        LoadFromCsv(fileName);
        allEvents = events.ToList();
        currentView = allEvents.ToList();
        Clear_Datagrid();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Failed to load data.csv\n\n" +
            "Load Error",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error
        );
    }
}
```

Rysunek 2 Czytanie danych z CSV

```
private void LoadFromCsv(string fileName)
{
    events.Clear();

    var lines = File.ReadAllLines(fileName).Skip(1);

    foreach (var line in lines)
    {
        var parts = line.Split(';');

        var ev = new Zdarzenie
        {
            Title = parts[1],
            Date = DateTime.Parse(parts[2]),
            TimeStart = parts[3],
            TimeEnd = parts[4],
            Location = parts[5],
            Type = parts[6],
            Cost = decimal.Parse(parts[7]),
            Description = parts[8],
            Tags = parts[9]
                .Split(',')
                .Select(t => t.Trim())
                .Where(t => t.Length > 0)
                .ToList()
        };

        events.Add(ev);
    }
}
```

Po otrzymaniu danych `Clear_Datagrid` odnawia dane w tabeli (On jest używany wszędzie, gdzie są odnawiane dane).

1.2 Algorytmy zachowania danych do CSV

Na początku został opisany główny sposób odczytania danych. Jest także sposób ich zapisania na później. Tą pracę wykonuje `SaveToCsv`, gdzie robi dokładnie taką samą funkcję co i odczytanie tylko naodwrot zapisując dane.

Rysunek 3 Funkcji odnowienia datagrid

```
public void RefreshGrid()
{
    allEvents = events.ToList();
    dgvEvents.DataSource = null;
    dgvEvents.DataSource = events;
    currentView = allEvents.ToList();
}
7 references
private void Clear_Datagrid()
{
    RefreshGrid(); // Odnawia datagrid
    RefreshFilterOptions(); // Odnawia filtry
    ClearSortGlyphs(); // czyści strzałki sortowania
    UpdateRowColors(); // Zmienia kolory
    monthCalendar1.Visible = false;
    monthCalendar2.Visible = false;
    isSaved = false;
}
```

Rysunek 4. Zapis CSV

```
private void SaveToCsv(string fileName)
{
    using (StreamWriter sw = new StreamWriter(fileName))
    {
        sw.WriteLine("Id;Title;Date;TimeStart;TimeEnd;Location;Type;Cost;Description;Tags");

        foreach (var e in events)
        {
            string tags = string.Join(",", e.Tags);

            sw.WriteLine(
                $"{e.Id};{e.Title};{e.Date:yyyy-MM-dd};{e.TimeStart};{e.TimeEnd};" +
                $"{e.Location};{e.Type};{e.Cost};{e.Description};{tags}"
            );
        }
    }
    isSaved = true;
}
```

Program zawsze zachowując nowe dane, tworząc backup dla starych danych. Także przy zamykaniu programu on wyświetli komunikat z pytaniem Save table? Oto kod tego komunikatu:

Rysunek 5 Zapis przy zamknięciu

```
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    if (isSaved)
        return;

    DialogResult result = MessageBox.Show(
        "Save table?",
        "Confirm exit",
        MessageBoxButtons.YesNoCancel,
        MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        SaveToCsv("data.csv");
    }
    else if (result == DialogResult.Cancel)
    {
        e.Cancel = true;
    }
}
```

1.3 Algorytmy sortowania

Dla włączenia sortowania w tabeli na początku jest wywoływana funkcja co pozwoli na sortowanie tabeli.

Rysunek 6 Początek sortowania

```
foreach (DataGridViewColumn col in dgvEvents.Columns)
{
    col.SortMode = DataGridViewColumnSortMode.Programmatic;
}
// Sortowanie
```

W samej programie po kliknięciu zostaje wywołana funkcja `ApplySorting()`, która sortuje klasę sortowania i wypisuje dane do tabeli. Dla sortowania trzeba klikać na Header tabelki.

Rysunek 7 Sort

```
if (dataProperty == "Title")
    keySelector = e => e.Title;
else if (dataProperty == "DateTimeDisplay")
    keySelector = e => e.Date;
else if (dataProperty == "Location")
    keySelector = e => e.Location;
else if (dataProperty == "Type")
    keySelector = e => e.Type;
else if (dataProperty == "Cost")
    keySelector = e => e.Cost;

if (keySelector == null)
    return;

currentView = sortAscending
    ? currentView.OrderBy(keySelector).ToList()
    : currentView.OrderByDescending(keySelector).ToList();

dgvEvents.DataSource = null;
dgvEvents.DataSource = currentView;
```

W programie jest także własna funkcja bubble sort, która sortuje po Nazwie wydarzenia i odpalić jej można przez Ctrl + b.

Rysunek 8 Bubble sort

```
private void BubbleSortByTitle(List<Zdarzenie> list)
{
    int n = list.Count;
    bool swapped;

    for (int i = 0; i < n - 1; i++)
    {
        swapped = false;

        for (int j = 0; j < n - i - 1; j++)
        {
            if (string.Compare(
                list[j].Title,
                list[j + 1].Title,
                StringComparison.OrdinalIgnoreCase) > 0)
            {
                Zdarzenie temp = list[j];
                list[j] = list[j + 1];
                list[j + 1] = temp;

                swapped = true;
            }
        }

        // if no swap, list already sorted
        if (!swapped)
            break;
    }
}
```

Rysunek 9 GUI dla filtrowania

Filters

Event type

☐ Balet
☐ Debata
☐ Festiwal

Data range

YYYY/MM/DD
YYYY/MM/DD

Location

☐ Akademia Smaku
☐ Biblioteka
☐ Centrum Dialogu

Tag

☐ autor
☐ balet
☐ dramat

Apply Filters

Clear All Filters

1.4 Algorytmy filtrowania i wyszukiwania

Wybierając potrzebne filtry i klikniesz Apply Filters klasa dla filtrowania zostaje odfiltrowana i także ta informacja zostaje wyświetlona w tabelce.

Rysunek 10 Funkcja filtrowania

```
// Date filter
if (hasFrom)
    filtered = filtered.Where(e => e.Date.Date >= from.Date);

if (hasTo)
    filtered = filtered.Where(e => e.Date.Date <= to.Date);

// Type filter
if (selectedTypes.Any())
    filtered = filtered.Where(e => selectedTypes.Contains(e.Type));

// Location filter
if (selectedLocations.Any())
    filtered = filtered.Where(e => selectedLocations.Contains(e.Location));

// Tags filter
if (selectedTags.Any())
    filtered = filtered.Where(e =>
        e.Tags.Any(tag => selectedTags.Contains(tag)));

dgvEvents.DataSource = null;
dgvEvents.DataSource = filtered.ToList();
currentView = filtered.ToList();
```

Także kliknąwszy Clear All Filters, tabela zostaje przywrócona do oryginału.

Rysunek 11 Funkcja czyszczenia filtrów

```
private void Clear_Filters_Click(object sender, EventArgs e)
{
    DateStart.Clear();
    DateEnd.Clear();
    txtSearch.Clear();

    // Czysci filtry
    for (int i = 0; i < clbType.Items.Count; i++)
        clbType.SetItemChecked(i, false);

    for (int i = 0; i < clbLocation.Items.Count; i++)
        clbLocation.SetItemChecked(i, false);

    for (int i = 0; i < clbTags.Items.Count; i++)
        clbTags.SetItemChecked(i, false);

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = allEvents;

    currentView = allEvents.ToList();
    Clear_Datagrid();
}
```

Także można wyszukać po nazwie wydarzeń za pomocą wyszukiwarki. Wtedy zostaną wyświetlone wydarzenia z podobnymi nazwami.

Rysunek 12 seach GUI



Rysunek 13 Apply search

```
private void ApplySearch()
{
    string query = txtSearch.Text.Trim();

    IEnumerable<Zdarzenie> result = currentView ?? allEvents;

    if (!string.IsNullOrEmpty(query))
    {
        result = result.Where(e =>
            !string.IsNullOrEmpty(e.Title) &&
            e.Title.IndexOf(query, StringComparison.OrdinalIgnoreCase) >= 0);
    }

    currentView = result.ToList();

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = currentView;

    UpdateRowColors();
}
```

1.5 Inne algorytmy

W zawiera wiele innych funkcji i niektóre z nich zostaną opisane tutaj:

Delete Event kasuje kolumna:

Rysunek 14 Delete Event

```
1 reference
private void Delete_Event_Click(object sender, EventArgs e)
{
    if (dgvEvents.SelectedRows.Count > 0)
    {
        Zdarzenie selected = dgvEvents.SelectedRows[0].DataBoundItem as Zdarzenie;
        events.Remove(selected);
        Clear_Datagrid();
    }
}
```

Add Event przenosi użytkownika do form2, gdzie można stworzyć nowy event. Tam wypełniwszy wszystkie potrzebne kolumny można dodać nowy event. Edit Event przeniesi dane dla edytowania w form2.

Rysunek 15 Dodawanie/Edytowanie eventu

```
private void Newevent_Click(object sender, EventArgs e)
{
    if (editingEvent == null)
    {
        // ADD MODE
        NewEvent = new Zdarzenie();
        FillEvent(NewEvent);
    }
    else
    {
        // EDIT MODE
        FillEvent(editingEvent);
    }

    DialogResult = DialogResult.OK;
    Close();
}
```

Export day zapisze dane dnia do txt pliku :

Rysunek 16 Zapis txt

```
SaveFileDialog sfd = new SaveFileDialog
{
    Filter = "Text files (*.txt)|*.txt",
    FileName = $"Event_{ev.Date:yyyy-MM-dd}.txt"
};

if (sfd.ShowDialog() != DialogResult.OK)
    return;

using (StreamWriter sw = new StreamWriter(sfd.FileName))
{
    sw.WriteLine("Event details");
    sw.WriteLine(new string('-', 40));

    sw.WriteLine($"Title: {ev.Title}");
    sw.WriteLine($"Date: {ev.Date:yyyy-MM-dd}");
    sw.WriteLine($"Time: {ev.TimeStart} - {ev.TimeEnd}");
    sw.WriteLine($"Location: {ev.Location}");
    sw.WriteLine($"Type: {ev.Type}");
    sw.WriteLine($"Cost: {ev.Cost}");
    sw.WriteLine($"Description: {ev.Description}");
    sw.WriteLine($"Tags: {string.Join(", ", ev.Tags)}");
}
```

Check day otworzy nowy form, gdzie przeniesie potrzebne dane i wyświetli ich:

Rysunek 17 Check day

```
private void Check_day_Click(object sender, EventArgs e)
{
    if (dgvEvents.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select an event first.");
        return;
    }

    var selectedEvent =
        dgvEvents.SelectedRows[0].DataBoundItem as Zdarzenie;

    Day dayForm = new Day(selectedEvent, events, this);
    dayForm.Show();
}
```

1.6 Obsługa danych

Wszytkie dane tabeli są przechowywane w dwa sposoby: Baza danych w CSV i w klasie Zdarzenie.cs

Format jest taki:

Id;Title;Date;TimeStart;TimeEnd;Location;Type;Cost;Description;Tags (przez ',')

Przykład danych w CSV:

Oni są wczytywane na początku programu i zostają zachowane przy kliknięciu Ctrl + S albo kliknięciu Save przy wychodzeniu z programu.

Rysunek 18 CSV plik

```
Id;Title;Date;TimeStart;TimeEnd;Location;Type;Cost;Description;Tags
6;Koncert jazzowy;2026-02-05;18:00;20:00;Filharmonia;Muzyka;50;Wieczorny koncert jazzowy;muzyka,jazz,koncert
7;Wystawa malarstwa;2026-02-06;10:00;17:00;Galeria Sztuki;Wystawa;20;Wystawa wspolczesnego malarstwa;sztuka,wystawa,obrazy
8;Spektakl teatralny;2026-02-07;19:00;21:30;Teatr Miejski;Teatr;60;Klasyczny spektakl dramatyczny;teatr,dramat,sztuka
9;Warsztaty ceramiczne;2026-02-08;09:00;13:00;Dom Kultury;Warsztaty;30;Warsztaty tworzenia ceramiki;rekodzieio,warsztaty,sztuka
10;Pokaz filmowy;2026-02-09;17:00;19:00;Kino Centrum;Film;25;Pokaz filmu niezaleznego;film,kino,pokaz
11;Spotkanie autorskie;2026-02-10;16:00;18:00;Biblioteka;Literatura;0;Spotkanie z pisarzem;ksiazki,literatura,autor
12;Festiwal muzyczny;2026-02-11;14:00;23:00;Rynek Glowny;Festiwal;120;Calodniowy festiwal muzyczny;muzyka,festiwal,plener
13;Wuklad historyczny;2026-02-12;11:00;12:30;Uniwersytet;Wuklad;0;Wuklad o historii regionu;historia,edukacja,wuklad
```

Przykład danych w klasie:

Rysunek 19 Klasa

```
public class Zdarzenie
{
    private static int _nextId = 1;

    public int Id { get; private set; }
    public string Title { get; set; }
    public DateTime Date { get; set; }
    public string TimeStart { get; set; }
    public string TimeEnd { get; set; }
    public string Location { get; set; }
    public string Type { get; set; }
    public decimal Cost { get; set; }
    public string Description { get; set; }

    public List<string> Tags { get; set; }

    public Zdarzenie()
    {
        Id = _nextId++;
        Tags = new List<string>();
    }
}
```

Dane po odpaleniu programu są przechowywane tutaj i używane jako 3 klasy:

Rysunek 20 Inicjalizacja klasów

```
private List<Zdarzenie> events = new List<Zdarzenie>(); // Origin
public bool Event_Up=true;

private List<Zdarzenie> allEvents = new List<Zdarzenie>(); // Filtrowane
private List<Zdarzenie> currentView = new List<Zdarzenie>(); //Sortowanie
```

ROZDZIAŁ 2. PROJEKT SYSTEMU

Działanie wszystkich funkcji GUI będą przedstawione w tym rozdziale..

2.1 Początek działania programu

Po włączeniu programu od razu zostają odpalone wszystkie funkcje co znajdują się w Form1_Load().

Rysunek 21 Funkcja początku programu

```
private void Form1_Load(object sender, EventArgs e)
{
```

W tej funkcji od razu jest generowana tablica z potrzebnymi kolumnami:

Rysunek 22 Generacja tablicy

```
dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
{
    HeaderText = "Title",
    DataPropertyName = "Title"
});

dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
{
    HeaderText = "Date & Time",
    DataPropertyName = "DateTimeDisplay",
    Width = 180
});

dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
{
    HeaderText = "Location",
    DataPropertyName = "Location"
});

dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
{
    HeaderText = "Type",
    DataPropertyName = "Type"
});

dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
{
    HeaderText = "Cost",
    DataPropertyName = "Cost"
});
```

Poczym zostają zainicjalizowane sortowanie, style, obsługa skrótów klawiszowych i zostaje załadowane początkowe dane.

Rysunek 23 Inne inicjalizacje

```
// Generacja columnow
// Sortowanie
foreach (DataGridViewColumn col in dgvEvents.Columns)
{
    col.SortMode = DataGridViewColumnSortMode.Programmatic;
}
// Sortowanie
dgvEvents.EnableHeadersVisualStyles = false;

//Ladowanie daty na poczatku
LoadDataOnStartup();

//Enable obsługi skrotow klawiszowych
this.KeyPreview = true;
```

2.2 Działanie programu

Po wczytaniu danych można zobaczyć GUI programu:

Rysunek 24 Początek programu

Title	Date & Time	Location	Type	Cost
Koncert jazzowy	2026-02-05 18:00-20:00	Filharmonia	Muzyka	50
Wystawa malarstwa	2026-02-06 10:00-17:00	Galeria Sztuki	Wystawa	20
Spektakl teatralny	2026-02-07 19:00-21:30	Teatr Miejski	Teatr	60
Warsztaty ceramiczne	2026-02-08 09:00-13:00	Dom Kultury	Warsztaty	30
Pokaz filmowy	2026-02-09 17:00-19:00	Kino Centrum	Film	25
Spotkanie autorskie	2026-02-10 16:00-18:00	Biblioteka	Literatura	0
Festiwal muzyczny	2026-02-11 14:00-23:00	Rynek Główny	Festiwal	120
Wykład historyczny	2026-02-12 11:00-12:30	Uniwersytet	Wykład	0
Koncert rockowy	2026-02-13 19:00-22:00	Hala Sportowa	Muzyka	80
Targi książki	2026-02-14 10:00-18:00	Centrum Expo	Targi	15
Plener fotograficzny	2026-02-15 08:00-12:00	Park Miejski	Fotografia	0
Warsztaty taneczne	2026-02-16 17:00-20:00	Studio Tańca	Taniec	40
Pokaz baletowy	2026-02-17 18:30-20:30	Opera	Balet	90
Debate społeczna	2026-02-18 15:00-17:00	Centrum Dialogu	Debate	0

Na zielono jest zamalowana kolumna co jest teraz używana.

Rysunek 25 Zielony

Festiwal muzyczny	2026-02-11 14:00-23:00	Rynek Główny	Festiwal	120
-------------------	------------------------	--------------	----------	-----

Na żółto są zamalowane kolumny co mają nakładający się czas.

Rysunek 26 Żółty

Pokaz filmowy	2026-02-09 17:00-19:00	Kino Centrum	Film	25
Wystawa	2026-02-09 18:00-20:00	Polska	Wystawa	40
Spotkanie autorskie	2026-02-10 16:00-18:00	Biblioteka	Literatura	0

Na czerwono jest zamalowana kolumna z eventem co już był.

Rysunek 27 Czerwony

Title	Date & Time	Location	Type	Cost
Festiwal	2026-01-01 10:00-12:00	Polska	Festiwal	0

Klikając na Header kolumny można sortować informację:

Rysunek 28 Sortowanie

Title	Date & Time	Location	Type	Cost
Debate społeczna	2026-02-18 15:00-17:00	Centrum Dialogu	Debate	0
Festiwal	2026-01-01 10:00-12:00	Polska	Festiwal	0
Festiwal filmowy	2026-02-24 11:00-22:00	Kino Studyjne	Festiwal	150

Używając Ctrl + b można wysortować dane po nazwie

Używanie filtrów, żeby użyć naciśnij *Apply Filters*, żeby skasować filtry użyj *Clear All Filters*:

Rysunek 29 Filtrowanie

Filters

Event type

☐ Balet

☐ Debata

☐ Festiwal

Date range

2026-01-15

2026-02-19

Location

☐ Akademia Smaku

☐ Biblioteka

☐ Centrum Dialogu

Tag

☒ autor

☐ balet

☐ dramat

Apply Filters

Import new calendar

Save calendar

Search

Title	Date & Time	Location	Type	Cost
Spotkanie autorskie	2026-02-10 16:00-18:00	Biblioteka	Literatura	0
Wystawa	2026-02-09 18:00-20:00	Polska	Wystawa	40

Wpisując do wyszukiwarki można znaleźć podobne po nazwie wydarzenia:

Rysunek 30 Searching

Import new calendar

Save calendar

al

Search

Title	Date & Time	Location	Type	Cost
Wystawa malarstwa	2026-02-06 10:00-17:00	Galeria Sztuki	Wystawa	20
Spektakl teatralny	2026-02-07 19:00-21:30	Teatr Miejski	Teatr	60
Festiwal muzyczny	2026-02-11 14:00-23:00	Rynek Główny	Festiwal	120
Pokaz baletowy	2026-02-17 18:30-20:30	Opera	Balet	90
Festiwal filmowy	2026-02-24 11:00-22:00	Kino Studyjne	Festiwal	150
Festiwal	2026-01-01 10:00-12:00	Polska	Festiwal	0

Klikając na add Event nas przenosi na inny form, gdzie można wpisywać dane dla wydarzenia i poczym to wydarzenie dodać:

Rysunek 31 Add Event

Form2

Add event

Event title

Data

Time Start

Time End

Location

Type

Tags

Cost

Back

Add event

Klikając na Edit Event nas przenosi na inny form, gdzie można zmieniać dane dla wydarzenia:

Rysunek 32 Edit event

The screenshot shows a window titled 'Form2' with a header bar containing the text 'Add event' in a blue box. Below the header, there is a form with several input fields and a text area. The fields are labeled 'Event title', 'Data', 'Time Start', 'Time End', 'Location', 'Type', 'Tags', and 'Cost'. The text area is labeled 'Wykład o historii regionu'. At the bottom right of the form, there are two buttons: 'Back' and 'Add event'.

Event title	Wykład historyczny
Data	2026-02-12
Time Start	11:00
Time End	12:30
Location	Uniwersytet
Type	Wykład
Tags	historia, edukacja, wykład
Cost	0

Wykład o historii regionu

Back Add event

Używając Ctrl + S można zachować dane:

Używając Ctrl + E można wczytać dane:

Lub można także używać przycisków:

Rysunek 33 Import/Save btn

Import new calendar

Save calendar

Rysunek 34 Date Saved Succ

The screenshot shows a dialog box with a close button (X) in the top right corner. The text inside the dialog box reads 'Data saved successfully.' Below the text, there is an 'OK' button.

×

Data saved successfully.

OK

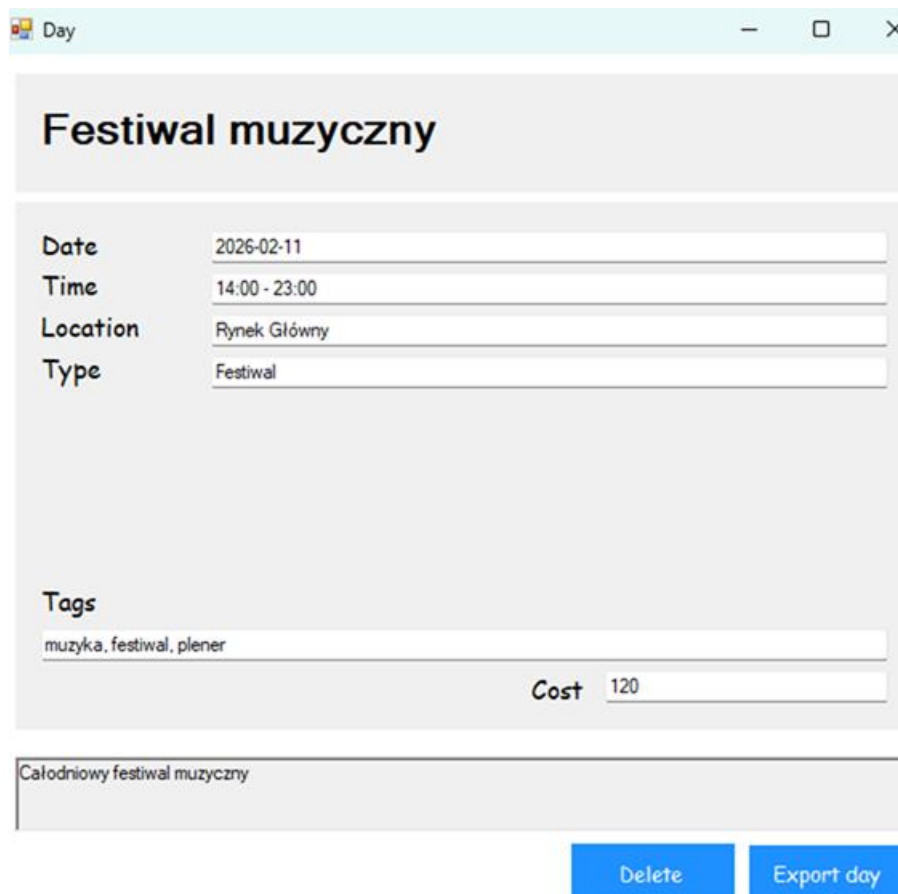
Export day pozwala na wypisanie planu dnia do txt pliku:

Rysunek 35 Txt day

```
Event details
-----
Title: Festiwal muzyczny
Date: 2026-02-11
Time: 14:00 - 23:00
Location: Rynek Główny
Type: Festiwal
Cost: 120
Description: Całodniowy festiwal muzyczny
Tags: muzyka, festiwal, plener
```

Lub można w samej programie to zobaczyć kliknawszy na Check day:

Rysunek 36 Check day btn



The screenshot shows a web application window titled "Day" with standard window controls. The main content area displays the title "Festiwal muzyczny" in a large, bold font. Below the title, there is a form with several input fields: "Date" (2026-02-11), "Time" (14:00 - 23:00), "Location" (Rynek Główny), and "Type" (Festiwal). Below these fields is a "Tags" section with a text input containing "muzyka, festiwal, plener". To the right of the tags is a "Cost" field with the value "120". At the bottom of the form, there is a text area containing "Całodniowy festiwal muzyczny". At the bottom right of the window, there are two blue buttons: "Delete" and "Export day".

ROZDZIAŁ 3. IMPLEMENTACJA

3.1 Opis kroków implementacji

Program został zaimplementowany w języku programowania C# używając dla tego technologii GUI Visual Studio 2019.

Najpierw był stworzo GUI, poczym dodano tabelkę > Zapis/Odczyt > Dodawanie/ usuwanie wydarzeń > Edytowanie wydarzeń > Day check/export > Filtrowanie > Sortowanie > Wyszukiwarka.

Także co zostało dodane można odnaleźć w GITHUBIE:

<https://github.com/DanielSuba/Kalendarz-zdarze-kulturalnych>

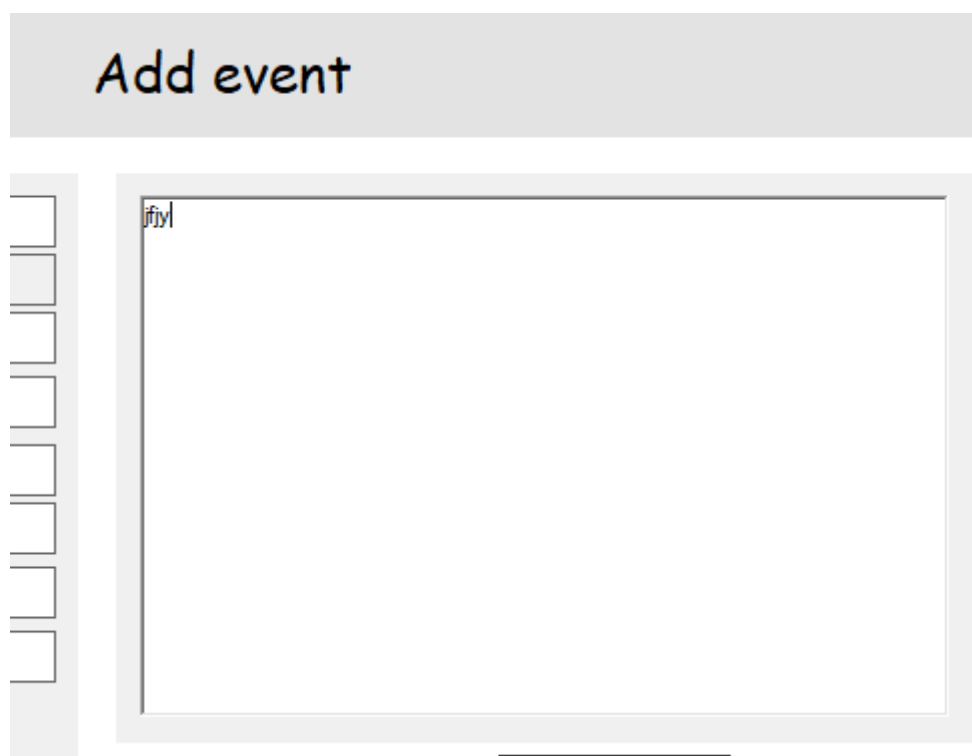
ROZDZIAŁ 4. TESTOWANIE I ANALIZA WYNIKÓW

4.1 Testy i ich wyniki

Program działa prędko i jest zoptymizowany jak tylko dałoby się.

W form2 z tej przyczyny, że są korzystana technologia CSV w description nie wolno używać Enter.

Rysunek 37 Description



Żeby ograniczyć stratę danych została dobawiona ochrona przed niechcącym wyjściem z programy:

Start new calendar

Save calendar

Title	Date & Time	Location	Type
koncert jazzowy	2026-02-05 18:00-20:00	Filharmonia	Muzyka
Wystawa malarstwa	2026-02-06 10:00-17:00	Galeria Sztuki	Wystawa
Teatr teatralny	2026-02-07 19:00-21:30	Teatr Miejski	Teatr
Warsztaty ceramiczne	2026-02-08 10:00-12:00	Centrum Kultury	Warsztaty
Film nowy	2026-02-09 18:00-20:00	Kinoteatr	Film
Wykład literacki	2026-02-10 19:00-21:00	Biblioteka	Literatura
Festiwal muzyczny	2026-02-11 14:00-18:00	Plac Wolności	Festiwal
Wykład historyczny	2026-02-12 10:00-12:00	Uniwersytet	Wykład
Koncert rockowy	2026-02-13 19:00-22:00	Hala Sportowa	Muzyka
Targi książki	2026-02-14 10:00-18:00	Centrum Expo	Targi
Fotograficzny	2026-02-15 08:00-12:00	Park Miejski	Fotografia
Tańce taneczne	2026-02-16 17:00-20:00	Studio Tańca	Taniec
Balet letowy	2026-02-17 18:30-20:30	Opera	Balet

ROZDZIAŁ 5. WNIOSKI

5.1 Ocena implementacji

Implementacja zajęła, około 7 dni, gdzie w ciągu tych dni byli dobawiane różne algorytmy. Także program był zarówno zmieniany z różnych przyczyn stylistycznych lub programowych. Dla zmniejszenia ilości błędów `Clear_Datagrid()` ciągle tworzy nową tablicę. W końcu program został zoptymizowany i działa szybko i wydajnie. Program jest także zrobiony ze zrozumiałym GUI, dlatego nowe użytkownicy smogą im korzystać bez problemów.

ZAKOŃCZENIE

Podsumowując mogę powiedzieć, że program reprezentujący działanie kalendarza wydarzeń kulturalnych pracuje wydajnie i jak najlepiej. Mam nadzieję, że ta dokumentacja będzie przydatna dla innych użytkowników.

SPIS RYSUNKÓW

Rysunek 1 Load data on start	5
Rysunek 2 Czytanie danych z CSV	5
Rysunek 3 Funkcji odnowienia datagrid	Error! Bookmark not defined.
Rysunek 4 Zapis Csv	6
Rysunek 5 Zapis przy zamknięciu.....	7
Rysunek 6 Początek sortowania	7
Rysunek 7 Sort	8
Rysunek 8 Bubble sort.....	Error! Bookmark not defined.
Rysunek 9 GUI dla filtrowania.....	9
Rysunek 10 Funkcja filtrowania.....	10
Rysunek 11 Funkcja czyszczenia filtrów	10
Rysunek 12 seach GUI	11
Rysunek 13 Apply search.....	11
Rysunek 14 Delete Event	11
Rysunek 15 Dodawanie/Edytowanie eventu	12
Rysunek 16 Zapis txt.....	12
Rysunek 17 Check day	13
Rysunek 18 Csv plik.....	13
Rysunek 19 Klasa	14
Rysunek 20 Inicjalizacja klasów	14
Rysunek 21 Funkcja początku programu	15
Rysunek 22 Generacja tablicy	15
Rysunek 23 Inne inicjalizacje.....	16
Rysunek 24 Początek programu	16
Rysunek 25 Zielony.....	16

Rysunek 26 Żółty	17
Rysunek 27 Czerwony.....	17
Rysunek 28 Sortowanie.....	17
Rysunek 29 Filtrowanie.....	17
Rysunek 30 Searching	18
Rysunek 31 Add Event.....	18
Rysunek 32 Edit event.....	19
Rysunek 33 Import/Save btn	19
Rysunek 34 Date Saved Succ	19
Rysunek 35 Txt day.....	20
Rysunek 36 Check day btn.....	20
Rysunek 37 Description	22

ZAŁĄCZNIK: KOD PROGRAMU

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO;

namespace Kalendarz_zdarzen_kulturalnych
{
    public partial class Form1 : Form
    {
        public bool Event_Up=true;

        private List<Zdarzenie> events = new List<Zdarzenie>(); // Origin
        private List<Zdarzenie> allEvents = new List<Zdarzenie>(); // Filtrowane
        private List<Zdarzenie> currentView = new List<Zdarzenie>(); //Sortwanie

        private int currentSortColumnIndex = -1;
        private bool sortAscending = true;
        private bool isSaved = false;
        public Form1()
        {
            InitializeComponent();
            this.KeyPreview = true;
        }
        private void Form1_Load(object sender, EventArgs e)
        {
            DateStart.Text = "YYYY/MM/DD";
            DateStart.ForeColor = Color.Gray;
            DateEnd.Text = "YYYY/MM/DD";
            DateEnd.ForeColor = Color.Gray;
            dgvEvents.AutoGenerateColumns = false;

            // Generacja columnow

            dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
            {
                HeaderText = "Title",
                DataPropertyName = "Title"
            });

            dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
            {
                HeaderText = "Date & Time",
                DataPropertyName = "DateTimeDisplay",
                Width = 180
            });

            dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
            {
                HeaderText = "Location",
                DataPropertyName = "Location"
            });

            dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
            {
                HeaderText = "Type",
                DataPropertyName = "Type"
            });

            dgvEvents.Columns.Add(new DataGridViewTextBoxColumn()
            {
                HeaderText = "Cost",
                DataPropertyName = "Cost"
            });
        }
    }
}
```

```

        // Generacja columnow
        // Sortowanie
        foreach (DataGridViewColumn col in dgvEvents.Columns)
        {
            col.SortMode = DataGridViewColumnSortMode.Programmatic;
        }
        // Sortowanie
        dgvEvents.EnableHeadersVisualStyles = false;

        //Ladowanie daty na poczatku
        LoadDataOnStartup();

        //Enable obslugi skrotow klawiszowych
        this.KeyPreview = true;

        monthCalendar1.Visible = false;
        monthCalendar2.Visible = false;
    }

    private void textBox6_Enter(object sender, EventArgs e)
    {
        monthCalendar1.Location = new Point(DateStart.Left, DateStart.Bottom);
        monthCalendar1.Visible = true;
        monthCalendar1.BringToFront();

        if (DateStart.Text == "YYYY/MM/DD")
        {
            DateStart.Text = "";
            DateStart.ForeColor = Color.Black;
        }
    }

    private void textBox6_Leave(object sender, EventArgs e)
    {
        monthCalendar2.Location = new Point(DateEnd.Left, DateEnd.Bottom);
        monthCalendar2.Visible = true;
        monthCalendar2.BringToFront();

        if (DateEnd.Text == "YYYY/MM/DD")
        {
            DateEnd.Text = "";
            DateEnd.ForeColor = Color.Black;
        }
    }

    private void textBox7_Enter(object sender, EventArgs e)
    {
        if (DateEnd.Text == "YYYY/MM/DD")
        {
            DateEnd.Text = "";
            DateEnd.ForeColor = Color.Black;
        }
    }
}

// ***** CSV *****

protected override bool ProcessCmdKey(ref Message msg, Keys keyData)
{
    //Save
    if (keyData == (Keys.Control | Keys.S))
    {
        SaveToCsv("data.csv");
        MessageBox.Show("Data saved successfully.");
        return true;
    }
    //Load
    if (keyData == (Keys.Control | Keys.E))
    {
        LoadFromCsvWithBackup();
        return true;
    }
}

//Legacy Sort

```

```

        if (keyData == (Keys.Control | Keys.B))
        {
            SortByTitleBubble();
            return true;
        }

        return base.ProcessCmdKey(ref msg, keyData);
    }

    private void SaveToCsv(string fileName)
    {
        using (StreamWriter sw = new StreamWriter(fileName))
        {
            sw.WriteLine("Id;Title;Date;TimeStart;TimeEnd;Location;Type;Cost;Description;Tags");

            foreach (var e in events)
            {
                string tags = string.Join(",", e.Tags);

                sw.WriteLine(
                    $"{e.Id};{e.Title};{e.Date:yyyy-MM-dd};{e.TimeStart};{e.TimeEnd};" +
                    $"{e.Location};{e.Type};{e.Cost};{e.Description};{tags}"
                );
            }
        }
        isSaved = true;
    }

    private string GetNextBackupFileName()
    {
        int i = 1;
        while (File.Exists($"backup{i}.csv"))
            i++;

        return $"backup{i}.csv";
    }

    private void LoadFromCsvWithBackup()
    {
        OpenFileDialog ofd = new OpenFileDialog
        {
            Filter = "CSV files (*.csv)|*.csv",
            Title = "Select CSV file"
        };

        if (ofd.ShowDialog() != DialogResult.OK)
            return;

        // Backup existing data
        if (events.Count > 0)
        {
            string backupName = GetNextBackupFileName();
            SaveToCsv(backupName);
        }

        LoadFromCsv(ofd.FileName);
        Clear_Datagrid();
    }

    private void LoadFromCsv(string fileName)
    {
        events.Clear();

        var lines = File.ReadAllLines(fileName).Skip(1); // skip header

        foreach (var line in lines)
        {
            var parts = line.Split(';');

            var ev = new Zdarzenie
            {
                Title = parts[1],
                Date = DateTime.Parse(parts[2]),
                TimeStart = parts[3],

```

```

        TimeEnd = parts[4],
        Location = parts[5],
        Type = parts[6],
        Cost = decimal.Parse(parts[7]),
        Description = parts[8],
        Tags = parts[9]
            .Split(',')
            .Select(t => t.Trim())
            .Where(t => t.Length > 0)
            .ToList()
    };

    events.Add(ev);
}

private void LoadDataOnStartup()
{
    string fileName = "data.csv";

    if (!File.Exists(fileName))
        return;

    try
    {
        LoadFromCsv(fileName);
        allEvents = events.ToList();
        currentView = allEvents.ToList();
        Clear_Datagrid();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            "Failed to load data.csv\n\n" + ex.Message,
            "Load Error",
            MessageBoxButtons.OK,
            MessageBoxIcon.Error
        );
    }
}

private void EksportCal_Click(object sender, EventArgs e)
{
    SaveToCsv("data.csv");
    MessageBox.Show("Data saved successfully.");
}

private void Import_Click(object sender, EventArgs e)
{
    LoadFromCsvWithBackup();
}

// ***** CSV
*****

private void textBox7_Leave(object sender, EventArgs e)
{
    if (string.IsNullOrEmpty(DateEnd.Text))
    {
        DateEnd.Text = "YYYY/MM/DD";
        DateEnd.ForeColor = Color.Gray;
    }
}

private void Add_event_Click(object sender, EventArgs e)
{
    using (Form2 f2 = new Form2())
    {
        if (f2.ShowDialog() == DialogResult.OK)
        {
            events.Add(f2.NewEvent);
            Clear_Datagrid();
        }
    }
}

```

```

/// <summary>
/// //////////////////////////////////Update////////////////////////////////////
/// </summary>
public void RefreshGrid()
{
    allEvents = events.ToList();
    dgvEvents.DataSource = null;
    dgvEvents.DataSource = events;
    currentView = allEvents.ToList();
}
private void Clear_Datagrid()
{
    RefreshGrid(); // Odnawia datagrid
    RefreshFilterOptions(); // Odnawia filtry
    ClearSortGlyphs(); // czyści strzałki sortowania
    UpdateRowColors(); // Zmienia kolory
    monthCalendar1.Visible = false;
    monthCalendar2.Visible = false;
    isSaved = false;
}
/// <summary>
/// //////////////////////////////////Update////////////////////////////////////
/// </summary>

private void Check_day_Click(object sender, EventArgs e)
{
    if (dgvEvents.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select an event first.");
        return;
    }

    var selectedEvent =
        dgvEvents.SelectedRows[0].DataBoundItem as Zdarzenie;

    Day dayForm = new Day(selectedEvent, events, this);
    dayForm.Show();
}

// xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx To co nie potrzebne xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
private void textBox9_TextChanged(object sender, EventArgs e)
{
}

private void Upcoming_Events_Click(object sender, EventArgs e)
{
}

private void Past_Events_Click(object sender, EventArgs e)
{
}

private void pictureBox7_Click(object sender, EventArgs e)
{
}

private void Filt_Cost_TextChanged(object sender, EventArgs e)
{
}

private void pictureBox3_Click(object sender, EventArgs e)
{
}

private void Filt_Title_TextChanged(object sender, EventArgs e)
{
}

```



```

        sw.WriteLine($"Time: {ev.TimeStart} - {ev.TimeEnd}");
        sw.WriteLine($"Location: {ev.Location}");
        sw.WriteLine($"Type: {ev.Type}");
        sw.WriteLine($"Cost: {ev.Cost}");
        sw.WriteLine($"Description: {ev.Description}");
        sw.WriteLine($"Tags: {string.Join(", ", ev.Tags)}");
    }
}

private void Edit_Event_Click(object sender, EventArgs e)
{
    if (dgvEvents.SelectedRows.Count == 0)
    {
        MessageBox.Show("Please select an event first.");
        return;
    }

    var selectedEvent =
        dgvEvents.SelectedRows[0].DataBoundItem as Zdarzenie;

    using (Form2 f2 = new Form2(selectedEvent))
    {
        if (f2.ShowDialog() == DialogResult.OK)
        {
            Clear_Datagrid();
        }
    }
}

// ***** Export event TXT
*****

// text box date
private void DateStart_TextChanged(object sender, EventArgs e)
{
    monthCalendar1.Visible = true;
    monthCalendar1.BringToFront();
}

private void DateEnd_TextChanged(object sender, EventArgs e)
{
    monthCalendar2.Visible = true;
    monthCalendar2.BringToFront();
}

private void monthCalendar1_DateSelected(object sender, DateRangeEventArgs e)
{
    DateStart.Text = e.Start.ToString("yyyy-MM-dd");
    monthCalendar1.Visible = false;
}

private void monthCalendar2_DateSelected(object sender, DateRangeEventArgs e)
{
    DateEnd.Text = e.Start.ToString("yyyy-MM-dd");
    monthCalendar2.Visible = false;
}

private void Form1_Click(object sender, EventArgs e)
{
    monthCalendar1.Visible = false;
    monthCalendar2.Visible = false;
}

// text box date
// ***** Filters
*****

private void ApplyDateFilter()
{
    DateTime from, to;
    bool hasFrom = DateTime.TryParse(DateStart.Text, out from);
    bool hasTo = DateTime.TryParse(DateEnd.Text, out to);
}

```

```

var selectedTypes = clbType.CheckedItems.Cast<string>().ToList();
var selectedLocations = clbLocation.CheckedItems.Cast<string>().ToList();
var selectedTags = clbTags.CheckedItems.Cast<string>().ToList();

var filtered = allEvents.AsEnumerable();

// Date filter
if (hasFrom)
    filtered = filtered.Where(e => e.Date.Date >= from.Date);

if (hasTo)
    filtered = filtered.Where(e => e.Date.Date <= to.Date);

// Type filter
if (selectedTypes.Any())
    filtered = filtered.Where(e => selectedTypes.Contains(e.Type));

// Location filter
if (selectedLocations.Any())
    filtered = filtered.Where(e => selectedLocations.Contains(e.Location));

// Tags filter
if (selectedTags.Any())
    filtered = filtered.Where(e =>
        e.Tags.Any(tag => selectedTags.Contains(tag)));

dgvEvents.DataSource = null;
dgvEvents.DataSource = filtered.ToList();
currentView = filtered.ToList();
}

private void Apply_Filters_Click(object sender, EventArgs e)
{
    ApplyDateFilter();
    UpdateRowColors();
}

private void Clear_Filters_Click(object sender, EventArgs e)
{
    DateStart.Clear();
    DateEnd.Clear();
    txtSearch.Clear();

    // Czysci filtry
    for (int i = 0; i < clbType.Items.Count; i++)
        clbType.SetItemChecked(i, false);

    for (int i = 0; i < clbLocation.Items.Count; i++)
        clbLocation.SetItemChecked(i, false);

    for (int i = 0; i < clbTags.Items.Count; i++)
        clbTags.SetItemChecked(i, false);

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = allEvents;

    currentView = allEvents.ToList();
    Clear_Datagrid();
}

private void monthCalendar1_Leave(object sender, EventArgs e)
{
    monthCalendar1.Visible = false;
}

private void monthCalendar2_Leave(object sender, EventArgs e)
{
    monthCalendar2.Visible = false;
}

private void DateStart_Click(object sender, EventArgs e)
{
    monthCalendar1.Visible = true;
}

```

```

        monthCalendar2.Visible = false;
    }

    private void DateEnd_Click(object sender, EventArgs e)
    {
        monthCalendar1.Visible = false;
        monthCalendar2.Visible = true;
    }

    private void Form1_MouseClick(object sender, MouseEventArgs e)
    {
        if (!DateStart.Bounds.Contains(e.Location) && !monthCalendar1.Bounds.Contains(e.Location))
        {
            monthCalendar1.Visible = false;
        }

        if (!DateEnd.Bounds.Contains(e.Location) && !monthCalendar2.Bounds.Contains(e.Location))
        {
            monthCalendar2.Visible = false;
        }
    }

    private void pictureBox1_Click(object sender, EventArgs e)
    {
        monthCalendar1.Visible = false;
        monthCalendar2.Visible = false;
    }

    private void RefreshFilterOptions()
    {
        clbType.Items.Clear();
        clbLocation.Items.Clear();
        clbTags.Items.Clear();

        var types = allEvents
            .Select(e => e.Type)
            .Where(t => !string.IsNullOrEmpty(t))
            .Distinct()
            .OrderBy(t => t);

        var locations = allEvents
            .Select(e => e.Location)
            .Where(l => !string.IsNullOrEmpty(l))
            .Distinct()
            .OrderBy(l => l);

        var tags = allEvents
            .SelectMany(e => e.Tags)
            .Where(t => !string.IsNullOrEmpty(t))
            .Distinct()
            .OrderBy(t => t);

        foreach (var t in types)
            clbType.Items.Add(t);

        foreach (var l in locations)
            clbLocation.Items.Add(l);

        foreach (var tag in tags)
            clbTags.Items.Add(tag);
    }

    // ***** Filters
    *****
    // ***** Sort
    *****

    private void dgvEvents_ColumnHeaderMouseClick(object sender, DataGridViewCellMouseEventArgs e)
    {
        if (currentView == null || currentView.Count == 0)
            return;

        if (currentSortColumnIndex == e.ColumnIndex)

```

```

        sortAscending = !sortAscending;
    else
    {
        currentSortColumnIndex = e.ColumnIndex;
        sortAscending = true;
    }

    ApplySorting();
}
private void ApplySorting()
{
    if (currentView == null || currentView.Count == 0)
        return;

    if (currentSortColumnIndex < 0)
        return;

    string dataProperty =
        dgvEvents.Columns[currentSortColumnIndex].DataPropertyName;

    Func<Zdarzenie, object> keySelector = null;

    if (dataProperty == "Title")
        keySelector = e => e.Title;
    else if (dataProperty == "DateTimeDisplay")
        keySelector = e => e.Date;
    else if (dataProperty == "Location")
        keySelector = e => e.Location;
    else if (dataProperty == "Type")
        keySelector = e => e.Type;
    else if (dataProperty == "Cost")
        keySelector = e => e.Cost;

    if (keySelector == null)
        return;

    currentView = sortAscending
        ? currentView.OrderBy(keySelector).ToList()
        : currentView.OrderByDescending(keySelector).ToList();

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = currentView;

    foreach (DataGridViewColumn col in dgvEvents.Columns)
        col.HeaderCell.SortGlyphDirection = SortOrder.None;

    dgvEvents.Columns[currentSortColumnIndex]
        .HeaderCell.SortGlyphDirection =
            sortAscending ? SortOrder.Ascending : SortOrder.Descending;

    UpdateRowColors();
}
private void ClearSortGlyphs()
{
    foreach (DataGridViewColumn col in dgvEvents.Columns)
        col.HeaderCell.SortGlyphDirection = SortOrder.None;
}

// ***** Sort
*****

// ***** Legacy bubble Sort
*****
private void BubbleSortByTitle(List<Zdarzenie> list)
{
    int n = list.Count;
    bool swapped;

    for (int i = 0; i < n - 1; i++)
    {
        swapped = false;

        for (int j = 0; j < n - i - 1; j++)
        {

```

```

        if (string.Compare(
            list[j].Title,
            list[j + 1].Title,
            StringComparison.OrdinalIgnoreCase) > 0)
        {
            Zdarzenie temp = list[j];
            list[j] = list[j + 1];
            list[j + 1] = temp;

            swapped = true;
        }
    }

    // if no swap, list already sorted
    if (!swapped)
        break;
}

private void SortByTitleBubble()
{
    if (currentView == null || currentView.Count == 0)
        return;

    BubbleSortByTitle(currentView);

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = currentView;

    ClearSortGlyphs();
    UpdateRowColors(); // zachowuje kolory
}

// ***** Legacy bubble Sort *****
// ***** Date colors *****

private DateTime GetEventEndDateTime(Zdarzenie e)
{
    TimeSpan end;
    TimeSpan.TryParse(e.TimeEnd, out end);
    return e.Date.Date.Add(end);
}

private bool TimesOverlap(Zdarzenie a, Zdarzenie b)
{
    TimeSpan aStart, aEnd, bStart, bEnd;

    TimeSpan.TryParse(a.TimeStart, out aStart);
    TimeSpan.TryParse(a.TimeEnd, out aEnd);
    TimeSpan.TryParse(b.TimeStart, out bStart);
    TimeSpan.TryParse(b.TimeEnd, out bEnd);

    return aStart < bEnd && bStart < aEnd;
}

private void UpdateRowColors()
{
    if (dgvEvents.Rows.Count == 0)
        return;

    var view = dgvEvents.DataSource as List<Zdarzenie>;
    if (view == null)
        return;

    DateTime now = DateTime.Now;

    foreach (DataGridViewRow row in dgvEvents.Rows)
        row.DefaultCellStyle.BackColor = Color.White;

    for (int i = 0; i < view.Count; i++)
    {
        if (GetEventEndDateTime(view[i]) < now)

```

```

        {
            dgvEvents.Rows[i].DefaultCellStyle.BackColor = Color.LightCoral;
        }
    }

    for (int i = 0; i < view.Count; i++)
    {
        if (GetEventEndDateTime(view[i]) < now)
            continue;

        for (int j = i + 1; j < view.Count; j++)
        {
            if (GetEventEndDateTime(view[j]) < now)
                continue;

            if (view[i].Date.Date != view[j].Date.Date)
                continue;

            if (TimesOverlap(view[i], view[j]))
            {
                if (dgvEvents.Rows[i].DefaultCellStyle.BackColor != Color.LightCoral)
                    dgvEvents.Rows[i].DefaultCellStyle.BackColor = Color.Khaki;

                if (dgvEvents.Rows[j].DefaultCellStyle.BackColor != Color.LightCoral)
                    dgvEvents.Rows[j].DefaultCellStyle.BackColor = Color.Khaki;
            }
        }
    }
}

// ***** Date colors
*****

// ***** Search button
*****

private void ApplySearch()
{
    string query = txtSearch.Text.Trim();

    IEnumerable<Zdarzenie> result = currentView ?? allEvents;

    if (!string.IsNullOrEmpty(query))
    {
        result = result.Where(e =>
            !string.IsNullOrEmpty(e.Title) &&
            e.Title.IndexOf(query, StringComparison.OrdinalIgnoreCase) >= 0);
    }

    currentView = result.ToList();

    dgvEvents.DataSource = null;
    dgvEvents.DataSource = currentView;

    UpdateRowColors();
}

private void txtSearch_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        ApplySearch();
        e.Handled = true;
        e.SuppressKeyPress = true;
    }
}

private void btnSearch_Click(object sender, EventArgs e)
{
    Clear_Datagrid();
    ApplySearch();
}

```

```

// ***** Search button
*****

// Zachowanie przy wyjściu

private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    if (isSaved)
        return;

    DialogResult result = MessageBox.Show(
        "Save table?",
        "Confirm exit",
        MessageBoxButtons.YesNoCancel,
        MessageBoxIcon.Question);

    if (result == DialogResult.Yes)
    {
        SaveToCsv("data.csv");
    }
    else if (result == DialogResult.Cancel)
    {
        e.Cancel = true;
    }
}
}
}

```